

IOException Handling in Java

1. Aim

To write and execute a Java program that demonstrates IOException by attempting to read from a FileReader after the stream has been closed, and to handle the exception using a try-catch block.

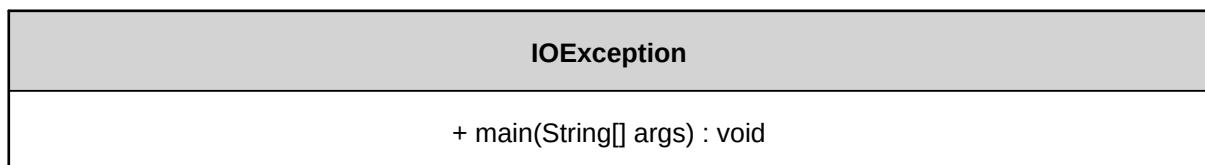
2. Description

IOException is a checked exception that occurs when an input or output operation fails. In this program, a FileReader is created for **test.txt**, then closed. The program attempts to read from the closed reader, causing an IOException that is handled by the catch block.

3. Algorithm

1. Start the program.
2. Create a FileReader for test.txt.
3. Close the FileReader.
4. Attempt to read from the closed reader.
5. Catch the IOException in the catch block.
6. Display the error message.
7. End the program.

4. UML Diagram



5. Source Code with Comments

```
import java.io.FileReader;
import java.io.IOException;

public class IOException {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("test.txt");
            reader.close();
            reader.read(); // Reading from a closed stream throws IOException
        } catch (IOException e) {
            System.err.println("Caught IOException: An I/O error occurred.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac IOException.java
java IOException
```

7. Output Screenshot

```
$ javac IOException.java  
$ java IOException  
Caught IOException: An I/O error occurred.
```

8. Result

The program was successfully compiled and executed. The IOException caused by attempting to read from a closed FileReader was successfully caught and handled.